

Parking Allotment Backend — Interview Quick Guide

FastAPI + PostgreSQL | Core Architecture, Concurrency & Defense Cheatsheet

1. System Concepts & Endpoints

Endpoint	Method	Purpose & Key Rule	Status
/slots	POST	Register slot with format <FLOOR>-<NUM> (e.g. A-1). Check duplicate.	201 / 409 / 422
/slots	GET	List slots ordered numerically (A-1 < A-2 < A-10 < B-1). Filter by status/type.	200 / 422
/vehicles	POST	Register vehicle plate (normalized uppercase), owner, and type.	201 / 409 / 422
/vehicles	GET	List vehicles with is_parked & current_slot (Single LEFT JOIN, no N+1).	200
/allocate	POST	Assigns best free slot based on vehicle preference & row lock.	201 / 400 / 409
/release	POST	Frees slot, marks RELEASED, calculates duration_minutes rounded up.	200 / 404 / 409
/allocations	GET	Lists active/released sessions sorted descending by entry time.	200 / 422

2. Core Engineering Decisions

Topic	The Problem	The Solution Implemented
Slot Ordering (§4.1)	Text sorting puts 'A-10' before 'A-2' (lexicographical bug).	Split into separate columns: floor (VARCHAR) + slot_num (INT) with composite B-Tree index.
Vehicle Preferences (§4.2)	Cars cannot take EV slots. EVs prefer EV slots but can fallback.	Preference list priority: BIKE → [BIKE], CAR → [CAR], EV → [EV, CAR].
Concurrency (§4.4)	Parallel allocations race for the same free slot.	1. Row lock: SELECT ... FOR UPDATE 2. DB Partial Unique Index: WHERE status = 'ACTIVE'.
No N+1 Queries (§5.4)	Fetching current slot per vehicle usually fires N extra queries.	Single query with LEFT OUTER JOIN on active allocations and slots.
Error Envelope (§5.8)	FastAPI default validation error leaks internals and non-standard JSON.	Global exception handlers returning uniform envelope: {error: {code, message, details}}.

3. Interview Defense: 'What Breaks If We Delete This Line?'

Be prepared to explain these exact critical lines during the technical review:

Code Snippet & Location	Technical Explanation & Failure Impact
<code>query = query.with_for_update() (app/services.py)</code>	Prevents concurrent race conditions. Without row locking, concurrent transactions read the same available slot simultaneously, causing double allocations or transaction rollback errors.
<code>CREATE UNIQUE INDEX ... WHERE status = 'ACTIVE' (schema.sql / models.py)</code>	Database-level single active allocation guarantee. If application checks fail or direct DB writes occur, the database acts as the ultimate authority rejecting duplicate active parkings.
<code>v = v.strip().upper() (app/schemas.py)</code>	Enforces plate normalization. Without this, 'mh12ab1234' and 'MH12AB1234' would be treated as two different vehicles, violating §4.6 uniqueness.
<code>math.ceil(delta_seconds / 60) (app/services.py)</code>	Duration rounding up. Guarantees any partial minute counts as a full minute per specification rules (e.g. 61 seconds = 2 minutes).
<code>order_by(Slot.floor.asc(), Slot.slot_num.asc()) (app/services.py)</code>	Guarantees §4.1 slot prioritization. Ensures A-2 is allotted before A-10 and floor A is prioritized before floor B.

4. Quick Reference Commands

Action	Command
Run Local Server	<code>python -m uvicorn app.main:app --reload</code>
Run Full Test Suite (TC01–TC14)	<code>python -m pytest -v</code>
Docker Compose Start	<code>docker compose up -d --build</code>
Swagger UI	<code>http://localhost:8000/docs</code>